

AI and Advanced Machine Learning

Module IX: Foundations of Generative Modeling II

Yan Kyaw Tun

Tenure Track Assistant Professor

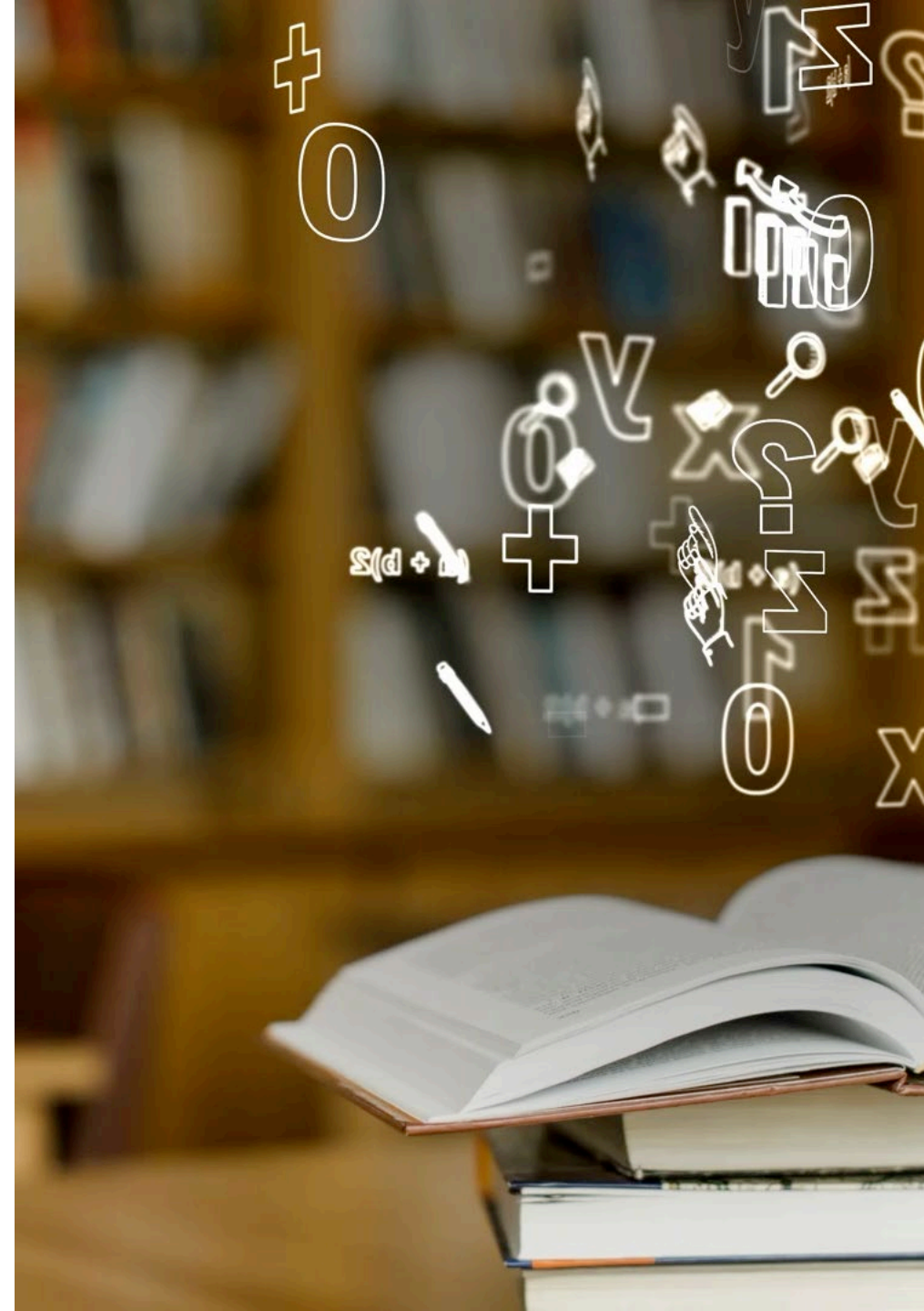
Edge Computing and Networking Group



AALBORG
UNIVERSITY

Outline

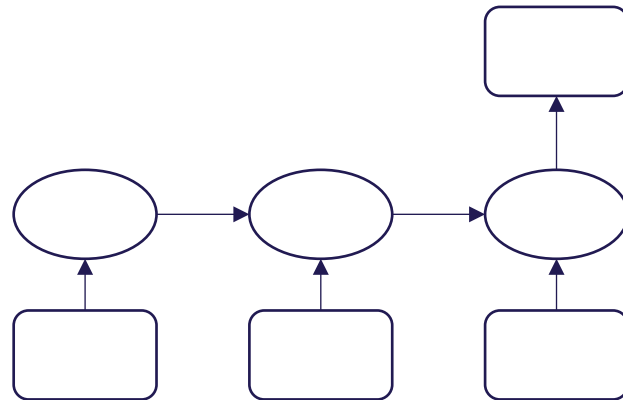
1. Sequence Modelling
 1. Introduction
 2. Attention mechanisms
2. «Attention is all you need»
 1. Architecture
 2. Building blocks
 3. Results and outlook



Sequence modelling

In sequence modelling we have a structured sequence of data $X = [x_0, x_1, x_2, \dots, x_n]$ which we wish to extract information from using a model f .

The task is to find a function $f(X)$ which can extract information $Y = [y_0, y_1, y_2, \dots, y_k]$ from the input sequence X .



Language

- Translation
- Sentiment Analysis
- Question Answering

Speech and Audio

- Speech Recognition
- Scene Classification
- Keyword Spotting

Video

- Activity Prediction
- Lip Reading

Attention Mechanisms

- Integral part of sequence modelling in various tasks
- Attention in simple terms
 - Generally, attention can be seen as the act of considering some inputs to be **more important** than others
 - It allows modelling of dependencies regardless of length of input or output sequences.
 - Example:
The chicken didn't cross the road because it was too ____
- By adding an attention mechanism, information (from each individual hidden state) can be summarized into a “context vector”.

Attention Mechanisms

- Original RNN Attention mechanism introduced by Bahdanau et al. (2015)
- "... allowing a model to automatically (soft-) search for parts of a source sentence that are relevant to predicting a target word ..."

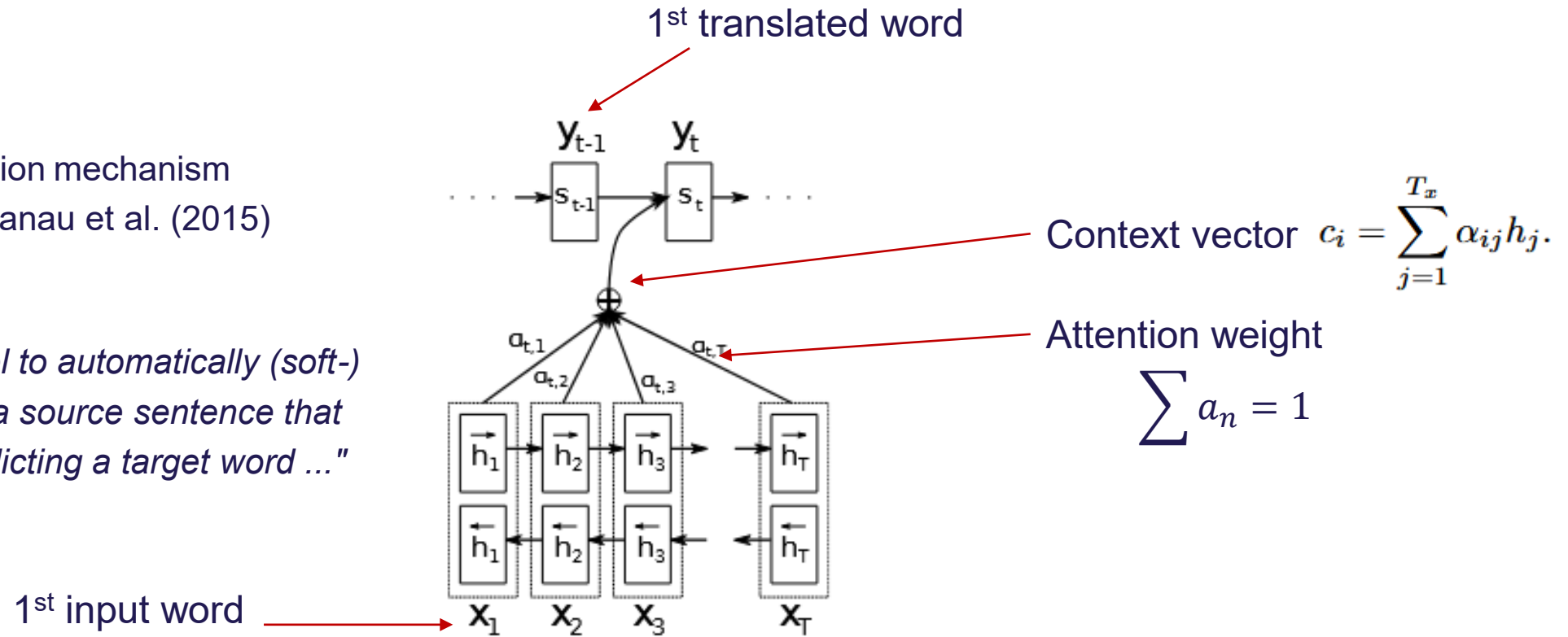


Figure 1: The graphical illustration of the proposed model trying to generate the t -th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.

Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.

Background

- In RNNs, the context information of the previous inputs still needs to be accumulated throughout the RNN states, i.e., the last state encodes all the information in a fixed size context vector.
 - *Problems?*
- Attention mechanisms solved some of the long-term dependency issues. However, they are computationally inefficient.
 - Each state relies on the previous hidden state.
To compute h_3 , h_2 must be computed first and so on

Can we build a model purely based on attention?

- Have all elements in the input sequence attend to all other elements through **self-attention**
- Contextual encoding of each element with respect to other elements

Attention is all you need

Attention Is All You Need

1. Background
2. Architecture
3. Building blocks
 1. Scaled dot-product attention
 2. Multi-head attention
 3. Feed forward network
 4. Residual / skip connections
 5. Positional encoding
4. Results

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

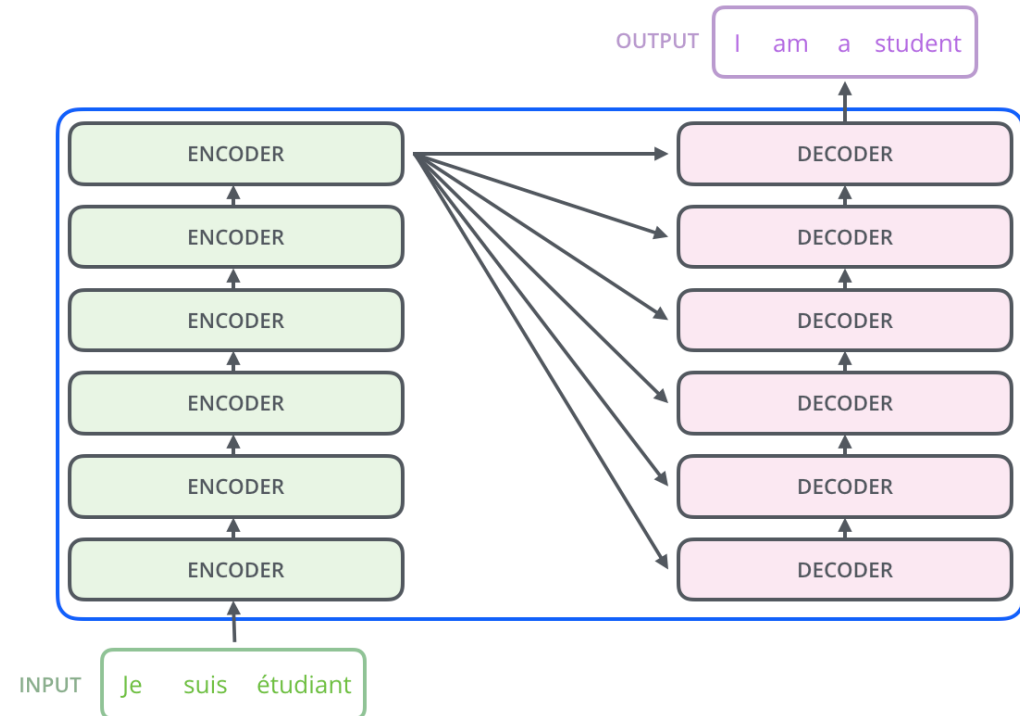
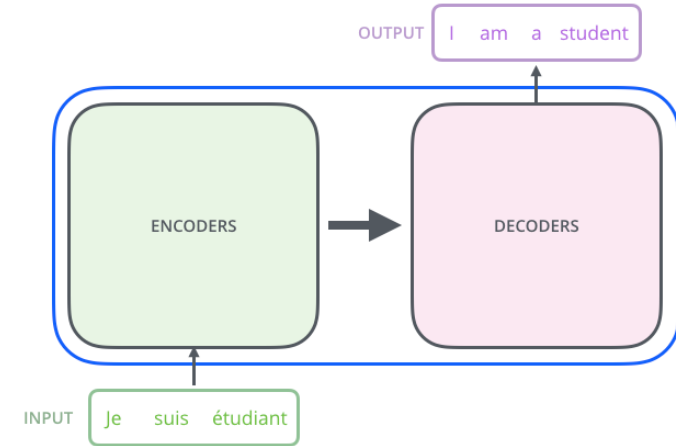
Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu	Lukasz Kaiser* Google Brain lukaszkaizer@google.com	
Illia Polosukhin* † illia.polosukhin@gmail.com			

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

Architecture

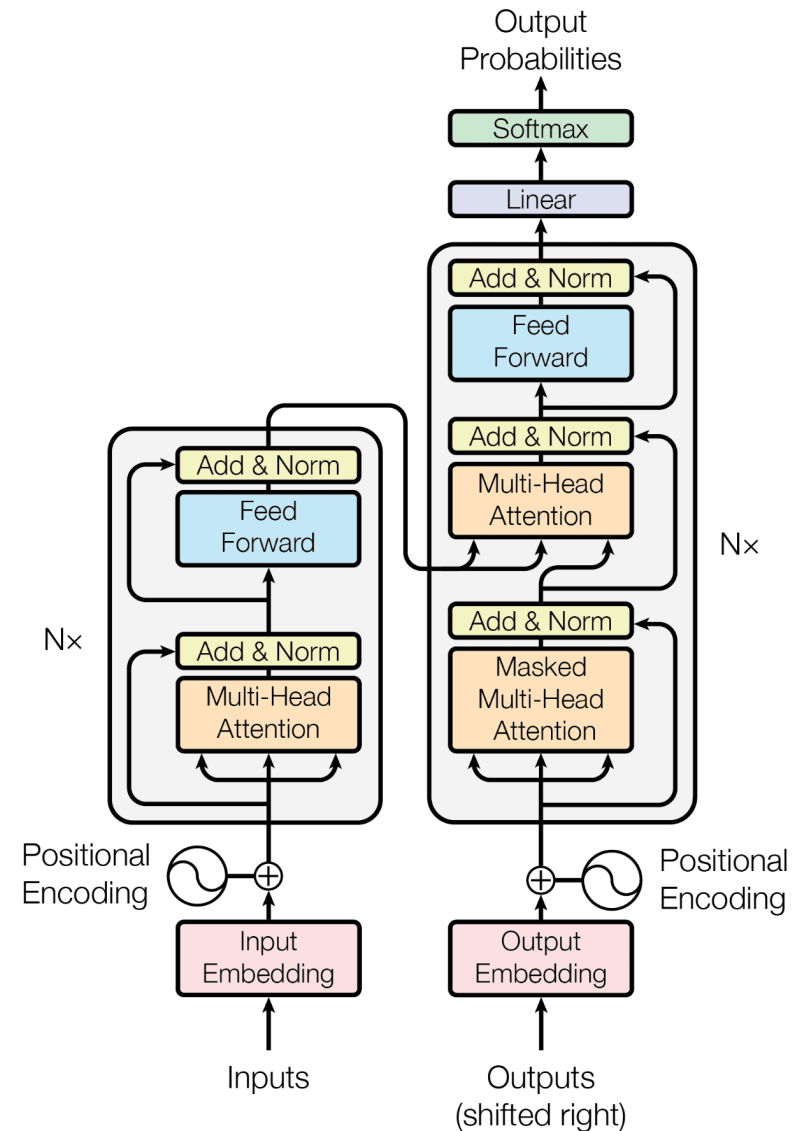
- **Encoder** maps an input sequence $(x_1, x_2, \dots, x_n)$ to representation $z = (z_1, z_2, \dots, z_n)$
- **Decoder** predicts output probabilities of the next sequence «auto-regressively» based on encoder representations and shifted output sequence.



Architecture

- **Encoder** maps an input sequence $(x_1, x_2, \dots, x_n)$ to representation $z = (z_1, z_2, \dots, z_n)$
- **Decoder** predicts output probabilities of the next sequence «auto-regressively» based on encoder representations and shifted output sequence.

$N = 6$



Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

Building blocks – Scaled dot-product attention

- Comparing each element of input sequence with **all other** elements to determine the relative importance of each sequence
- Focuses on **contextually** relevant information:
 - each word has a different vector that expresses different meanings depending on the surrounding words.
- Easily vectorized = parallelizable!

Building blocks – Scaled dot-product attention

► Contextual embedding:

The chicken didn't cross the road because **it** was too ____

What should be the properties of "it"?

The chicken didn't cross the road because it was too **tired**

The chicken didn't cross the road because it was too **wide**

Building blocks – Scaled dot-product attention

- High-level idea: instead of using x_i directly, we'll represent 3 separate roles each vector x_i plays: q, k, v .
- Scaled dot-product is a function that, given a query q and a set of key:value pairs (k, v) , determines a **weighted contribution** of values in the following steps:
 1. Compute a similarity of current element q with some prior element k
 2. *Scale* this product
 3. Apply a *softmax* function to obtain weights (positive and add up to 1)
 4. Dot product of these weights with values v

Building blocks – Scaled dot-product attention

- We use matrices to project each vector x_i into q, k, v:

Query: the *current element* being compared to preceding inputs

$$q_i = x_i W^Q$$

Key: a *preceding input* being compared to the current element to determine a similarity (dot product)

$$k_i = x_i W^K$$

Value: a value of a preceding element that is weighted and summed.

$$v_i = x_i W^V$$

Building blocks – Scaled dot-product attention

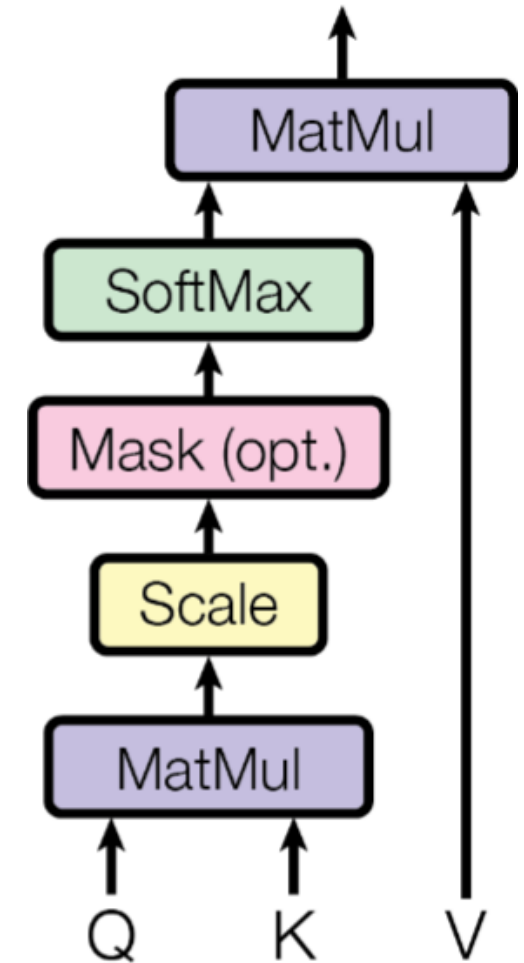
Math

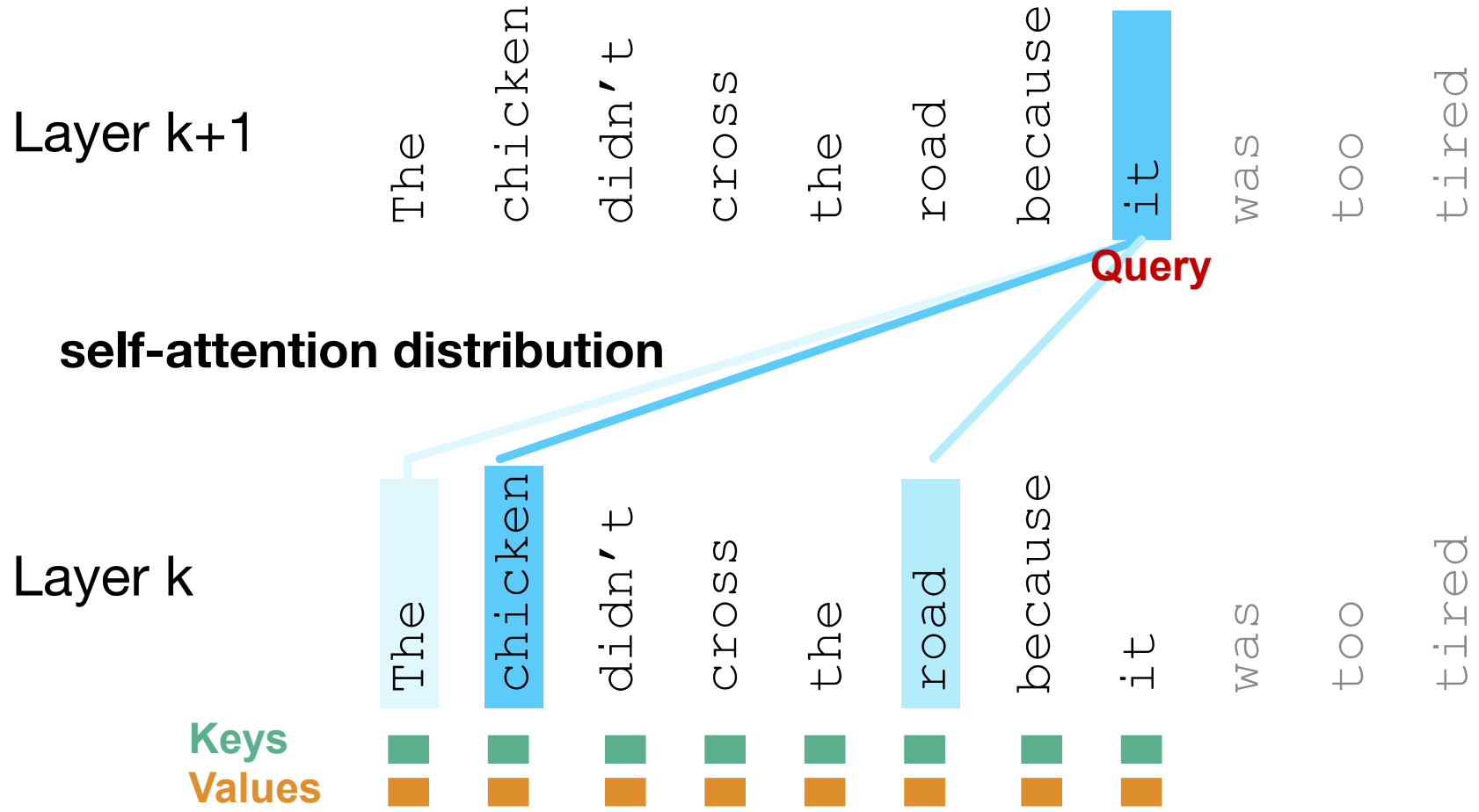
Input: queries $q \in \mathbb{R}^{d_k}$, keys $k \in \mathbb{R}^{d_k}$, and values $v \in \mathbb{R}^{d_v}$ are **vectors** of dimensions k, k, v respectively.

$$\text{Attn}: (\mathbb{R}^{d_k}, \mathbb{R}^{d_k}, \mathbb{R}^{d_v}) \rightarrow \mathbb{R}^{d_v},$$

$$A(Q, K, V) = \text{softmax}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) \cdot v$$

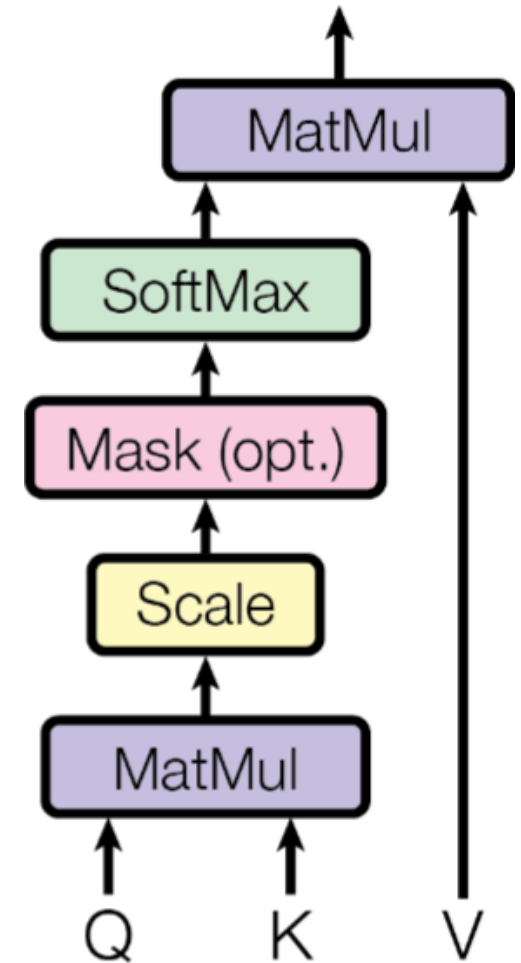
1. Dot product: $q \cdot k$
2. Scale the dot product: $w_v = \frac{q \cdot k}{\sqrt{d_k}}$
3. Apply softmax: $\text{softmax}\left(\frac{q \cdot k}{\sqrt{d_k}}\right)$
4. Get weighted values: $\text{softmax}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) \cdot v$





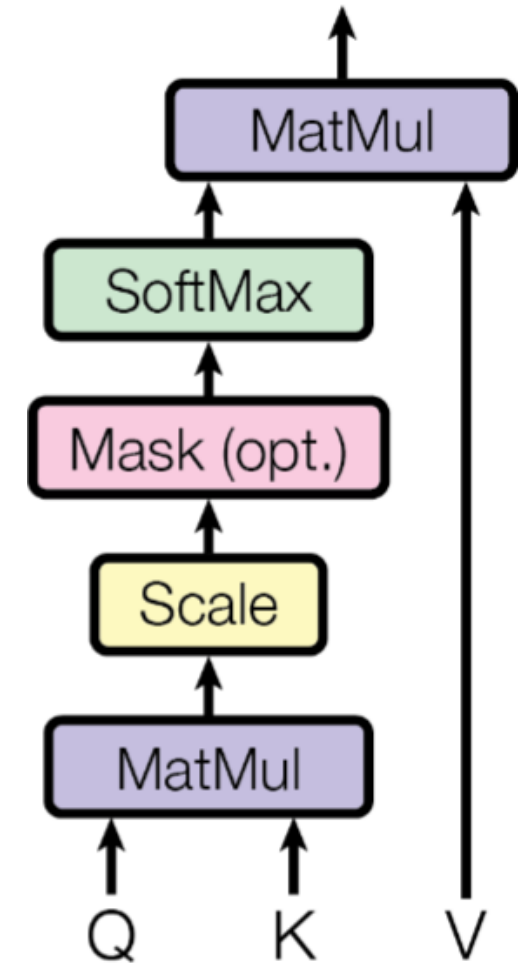
Building blocks – Scaled dot-product attention

- Why is it called self attention?
- Where is the learning??
- Why divide (scale) by $\sqrt{d_k}$?



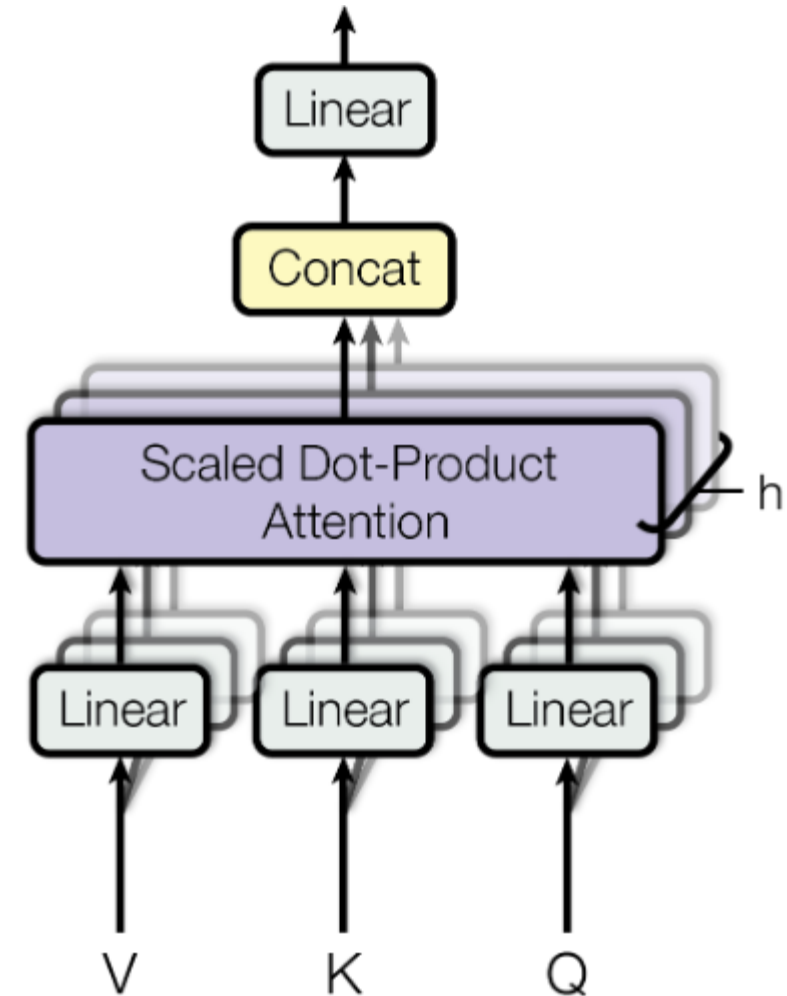
Building blocks – Scaled dot-product attention

- Why is it called self attention?
 - When queries, keys, and values are linear projections of the same input.
- Where is the learning??
 - We simply learn weights for q,k,v and compute attention on the linear projection.
- Why divide by $\sqrt{d_k}$?
 - Stop the magnitude of the dot product from growing too large.
 - Softmax! Softmax of large values -> very small gradients!



Building blocks – multi-head Attention

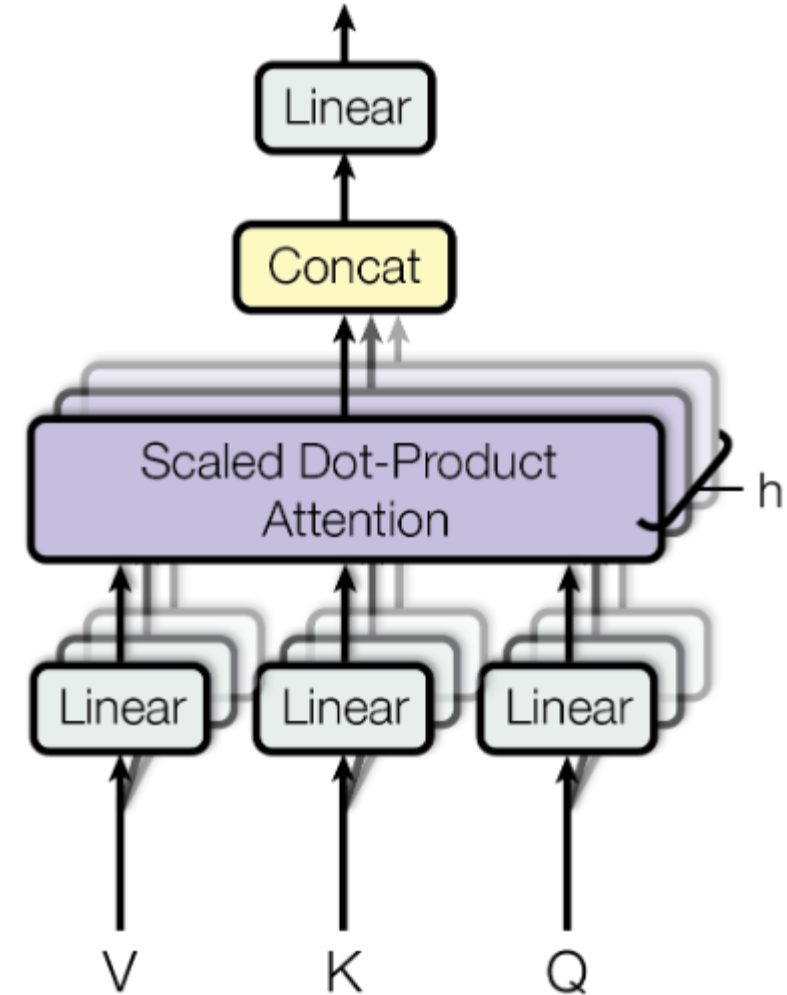
- ▶ Instead of computing attention once, how about we divide the workload across multiple **parallel attention computations**?
- ▶ Let d_m be the desired dimensionality of the feature space in which we want to compute attention.
- ▶ Instead of a single attention computation, we compute attention independently across h heads of dimensionality $\frac{d_m}{h}$



Building blocks – multi-head Attention

More specifically

1. Get $h, \frac{d_m}{h}$ dimensional linear projections of k, q, v
2. Compute self-attentions of these projections in parallel
3. Concatenate them to get a d_m dimensional aggregated output.
4. Apply a final linear projection.



Building blocks – multi-head Attention

$$MHA(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where

$$W_i^Q, W_i^K \in \mathbb{R}^{d_m \times d_k}$$

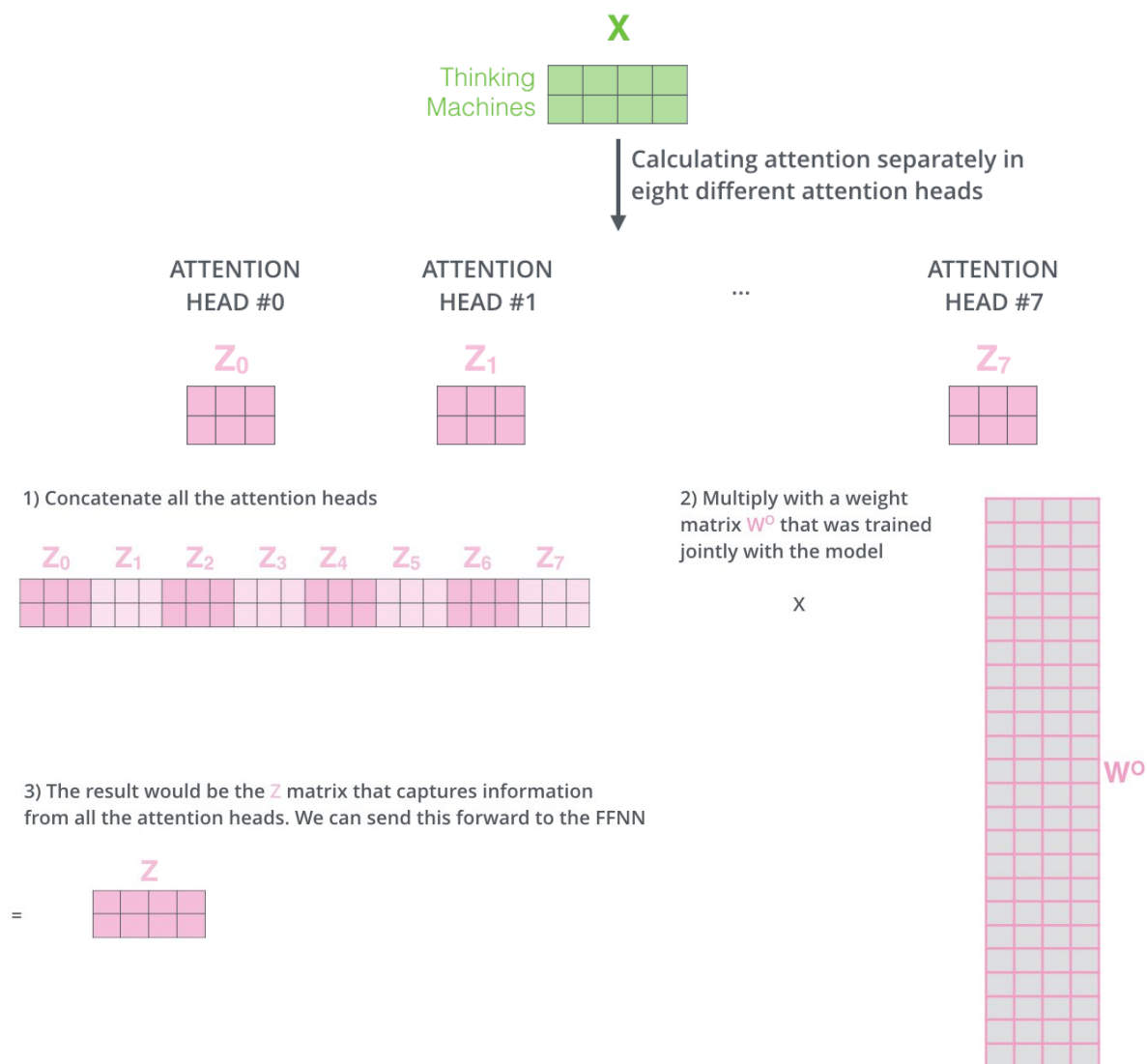
$$W_i^V \in \mathbb{R}^{d_m \times d_v}$$

$$W_i^O \in \mathbb{R}^{hd_v \times d_m}$$

Are learnable projections.

$$h = 8$$

$$d_k = d_v = \frac{d_m}{h} = 64$$



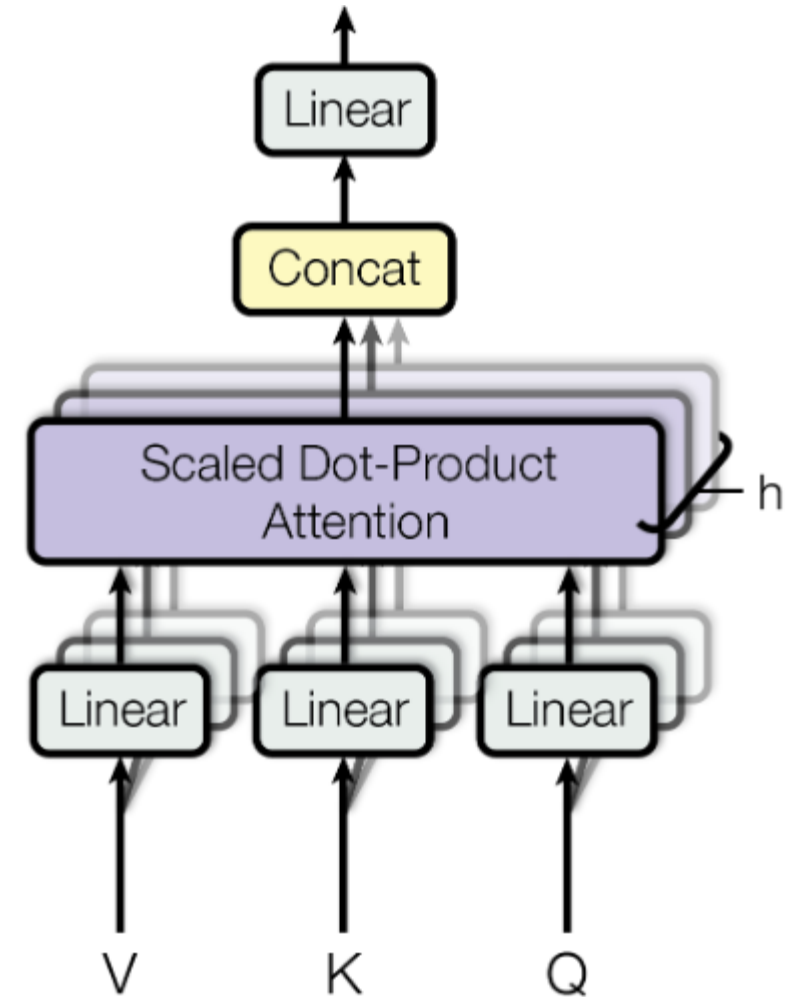
Building blocks – multi-head Attention

Why?

Empirically worked better.

Total computation is the same.

Different attention heads capture different information.



Building blocks – Summary of Attention

A method for enriching the representation of a token by incorporating contextual information.

The result: embedding for each words will be different in different context.

Contextual embedding: a representation of word meaning in its context.

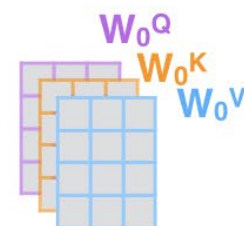
1) This is our input sentence*

Thinking
Machines

2) We embed each word*



3) Split into 8 heads. We multiply X or R with weight matrices



4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

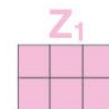
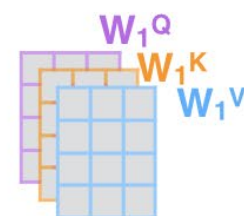
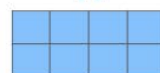


W^O



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

R



Building blocks – Feed forward network

Each layer has a FFN block after the MHA block:

$$FFN(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

A linear transformation → ReLU activation → linear transformation

$$x \in \mathbb{R}^{n \times d_m}$$

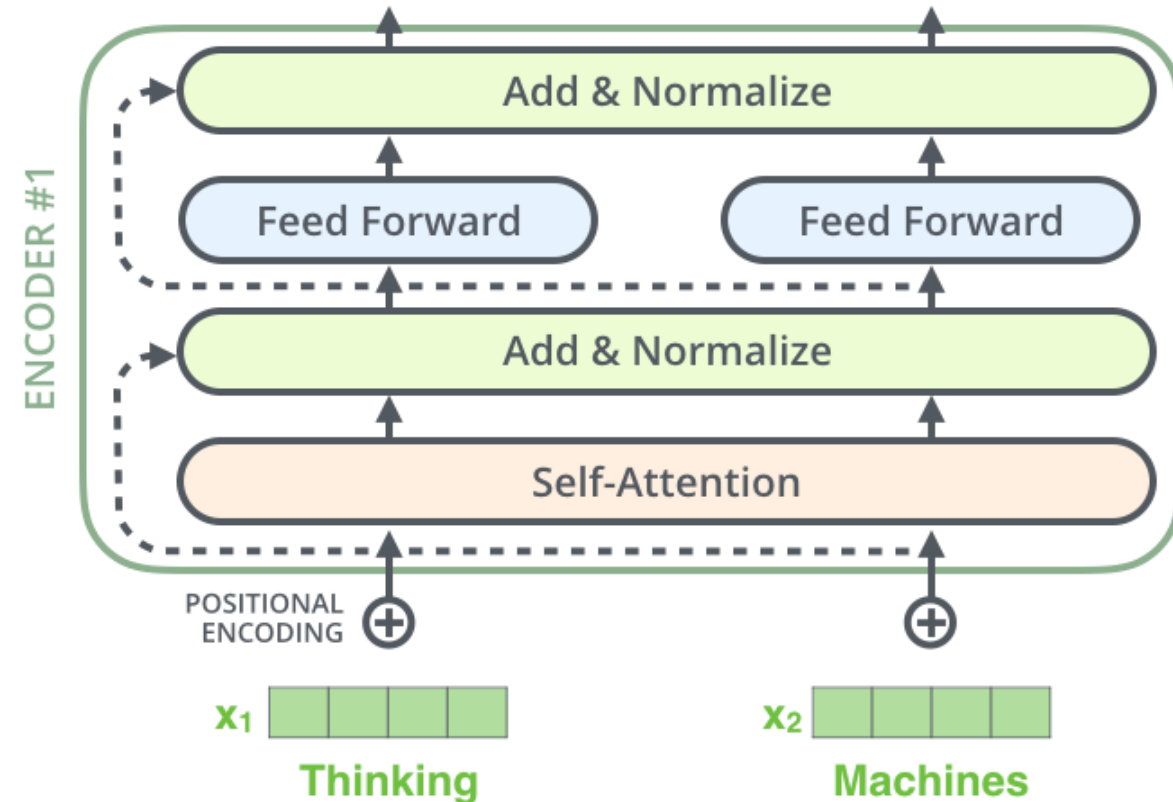
$$W_1 \in \mathbb{R}^{d_m \times d_h}$$

$$W_2 \in \mathbb{R}^{d_h \times d_m}$$

In the paper:

$$d_m = 512$$

$$d_h = 2048$$



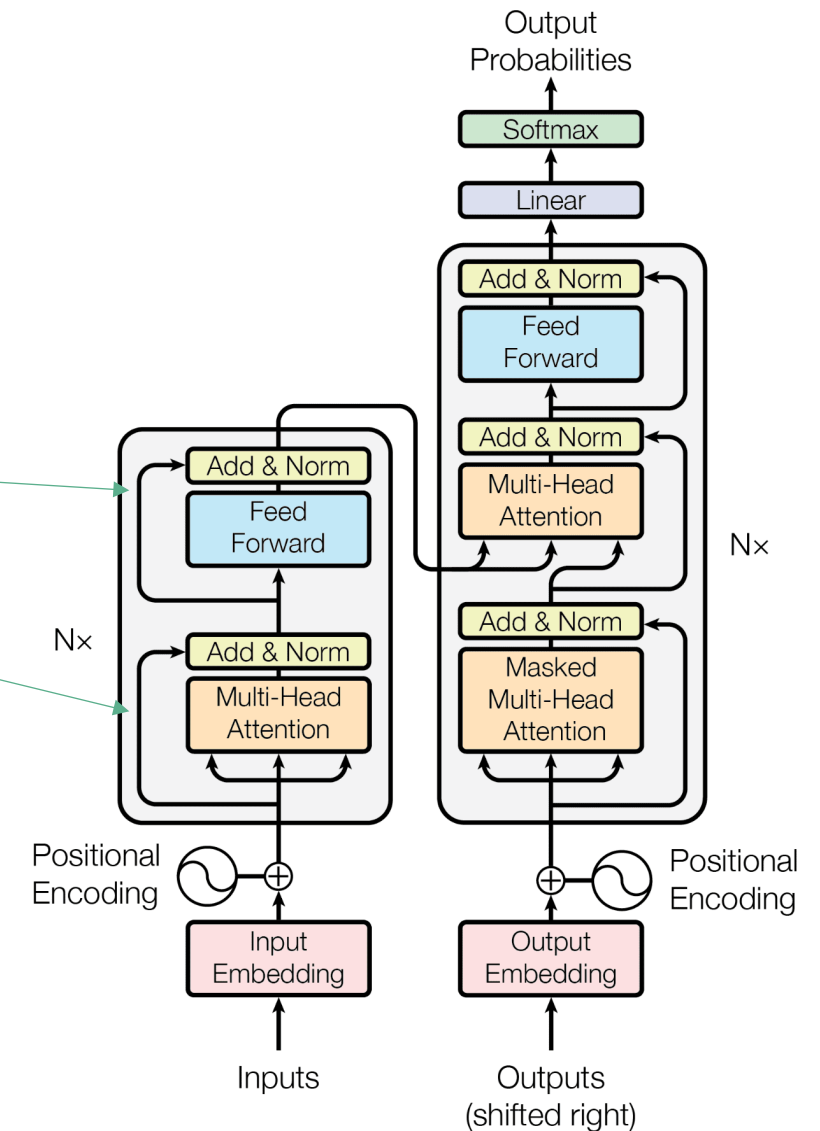
<https://jalammar.github.io/illustrated-transformer/>

Building blocks – Residual connection

Residual /skip connections made popular by ResNet

After every sub-block

- Help with gradient propagation (vanishing gradients)
- Allow training deeper networks without gradient related degradation

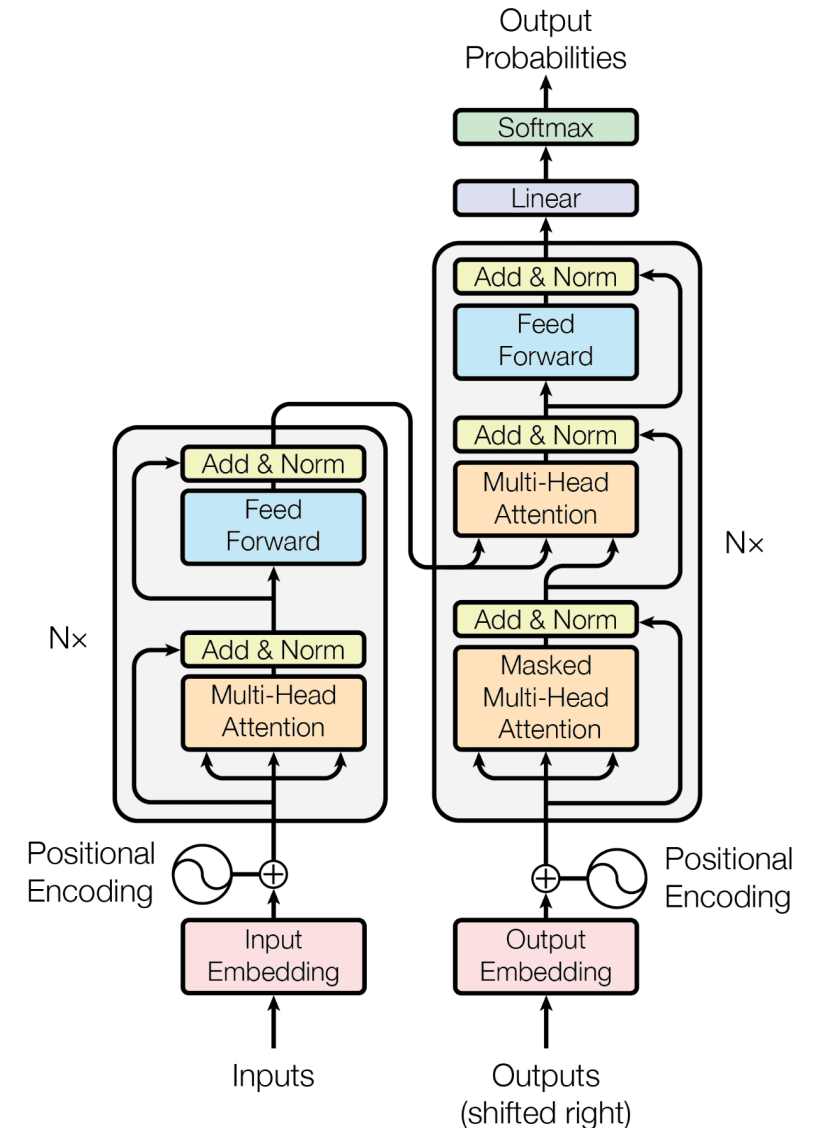


Building blocks – Positional encoding

- ▶ Inputs to transformers are tokens
- ▶ No convolution or RNN is done
- ▶ Everything is computed in parallel

How does the model know about word ordering?

We need to explicitly **encode positional information** into the input sequence!



Building blocks – Positional encoding

- ▶ Give the model some information about the **relative position** of the words in the sentence.
- ▶ Sinusoidal positional embedding:
- ▶ A vector $p \in \mathbb{R}^{d_m}$ such that

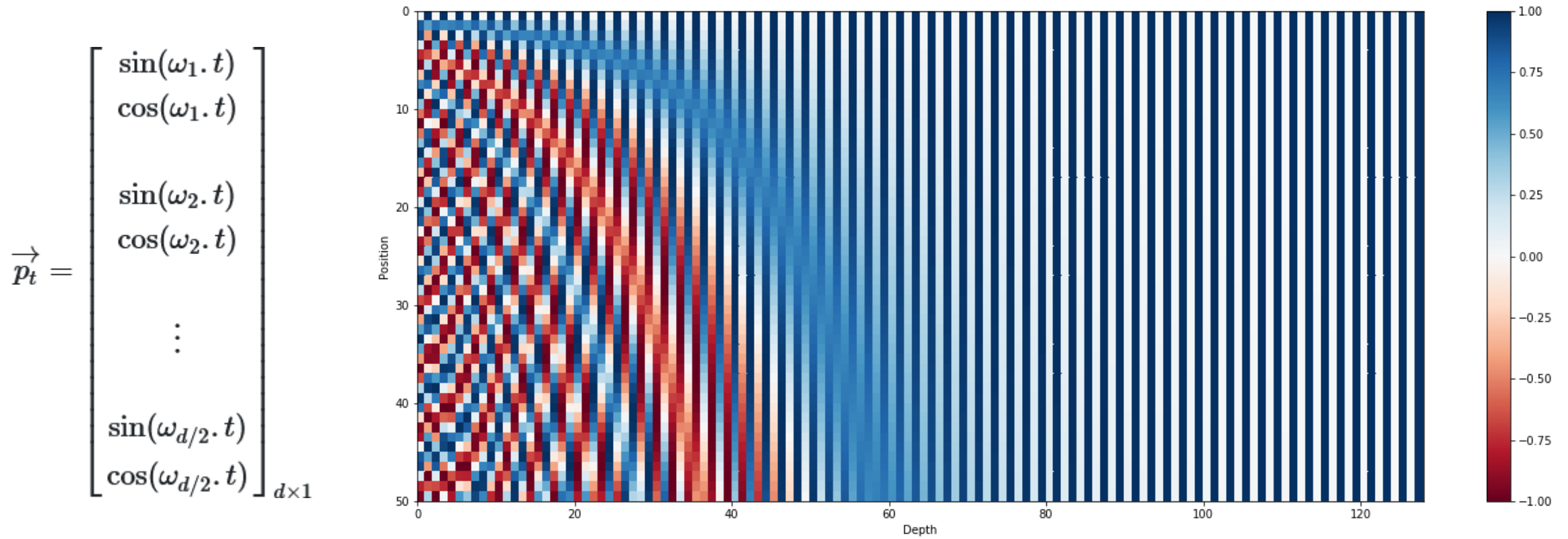
$$\begin{aligned}i &= 2k \\ i &= 2k + 1\end{aligned}$$

$$p[i](t) = \begin{cases} \sin(\omega_k \cdot t) \\ \cos(\omega_k \cdot t) \end{cases}$$

$$\text{Where } \omega_k = \frac{1}{10000^{2k/d_m}}$$

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

rate of change between different bits

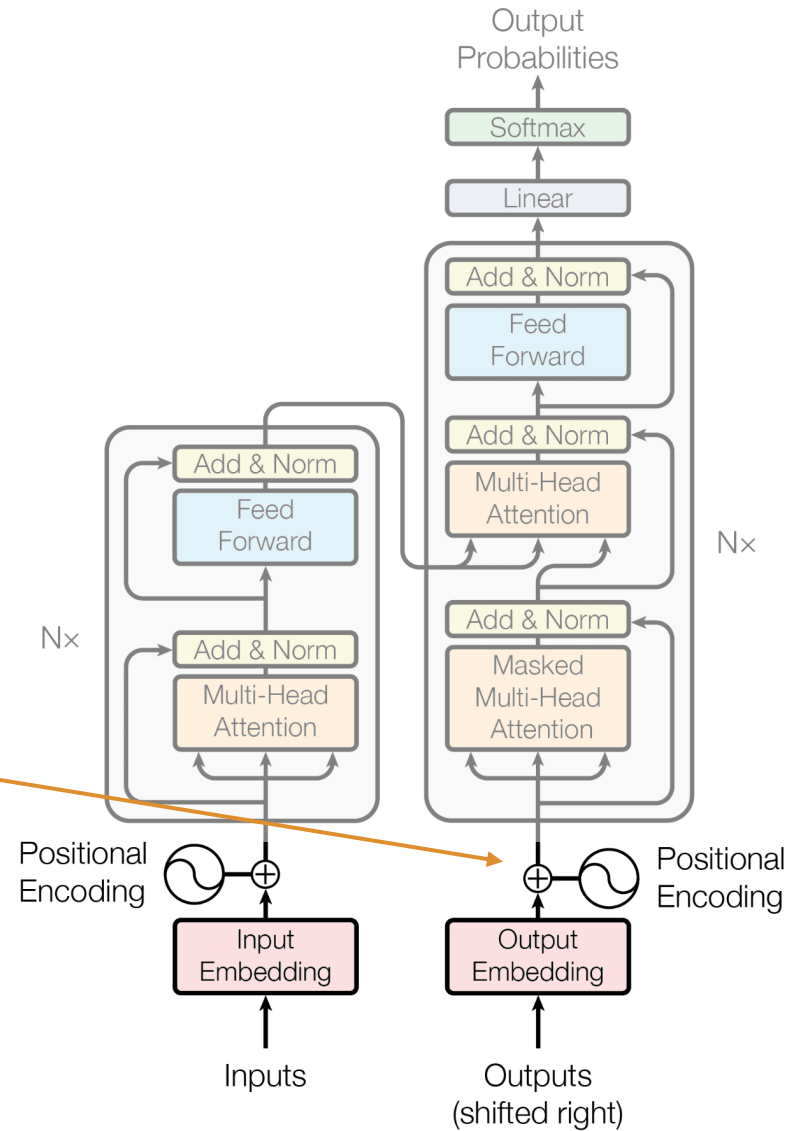


- Here positions range from 0-50, for an embedding dimension of 130.

Figure from: https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

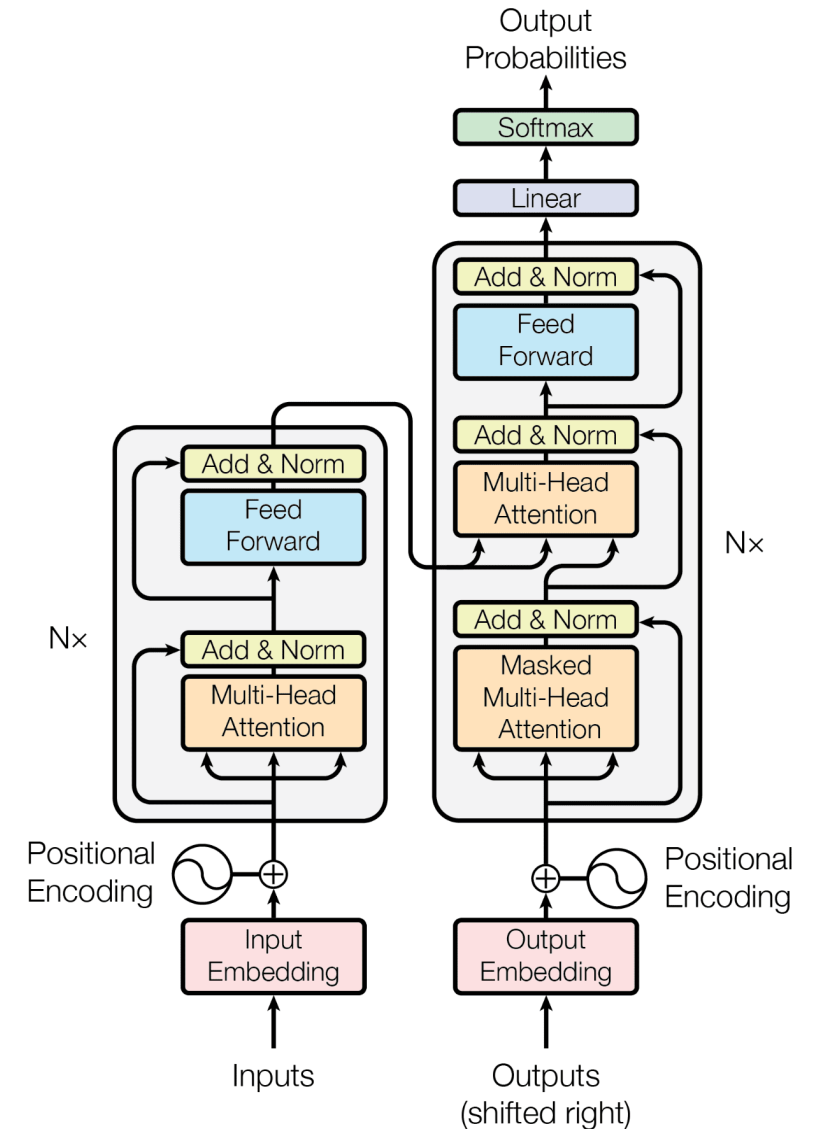
Building blocks – Positional encoding

- ▶ In other words, positional encoding is a vector with alternating sines and cosines
- ▶ $p_t = [\sin(\omega_1 t) \cos(\omega_1 t) \sin(\omega_2 t) \cos(\omega_2 t) \dots \sin(\omega_{d_{m/2}} t) \cos(\omega_{d_{m/2}} t)]^T$
- ▶ p_t is added to the input and output embeddings
- ▶ This provides unique vectors for each position.



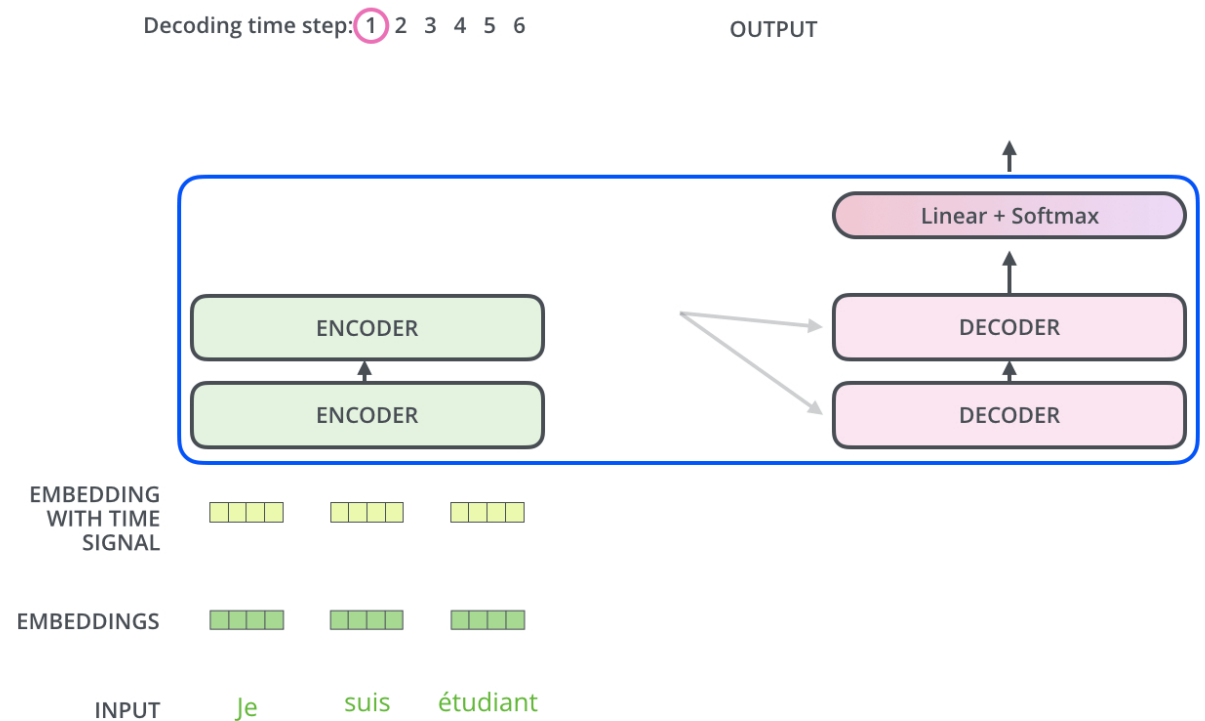
Building blocks – Positional encoding

- Why not use a learnable embedding?
 - Generalize to unseen lengths (extrapolate to longer sequences than those encountered at training times).
 - No extra parameters!



Decoder

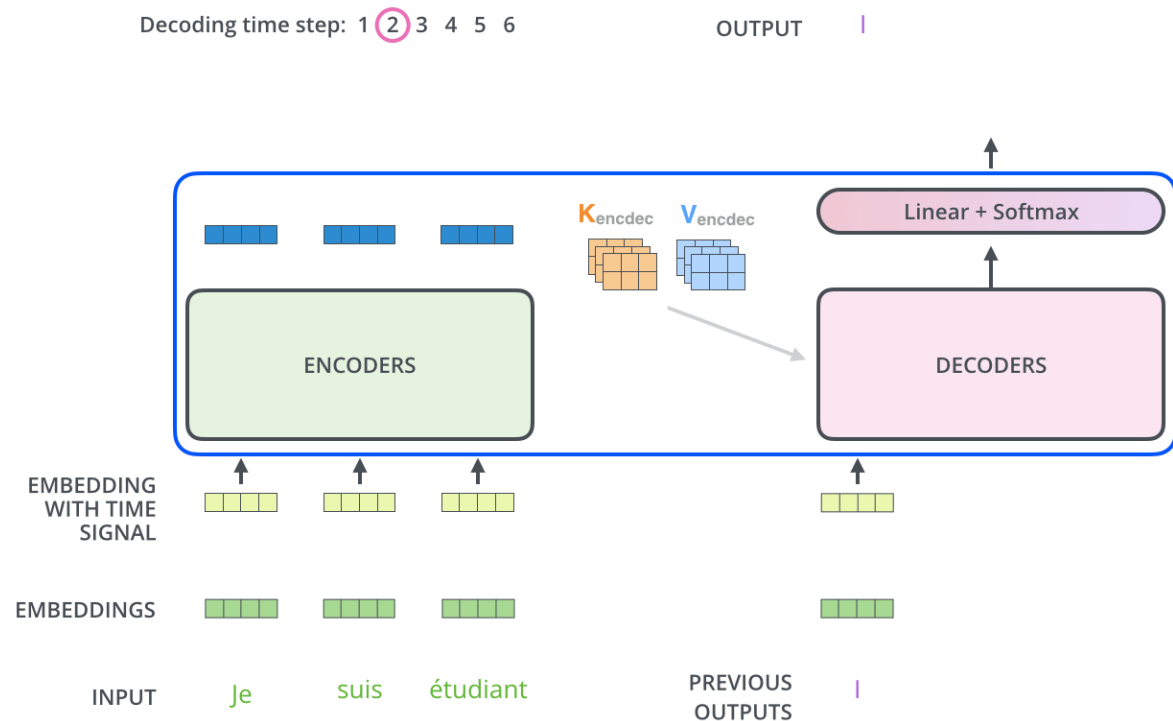
1. The encoder starts by processing the input sequence.
2. The output of the top encoder is transformed into a set of attention vectors K , V .
3. These are used by each decoder to predict the output probabilities for each token in the sequence.



<https://jalammar.github.io/illustrated-transformer/>

Decoder

1. The encoder starts by processing the input sequence.
2. The output of the top encoder is transformed into a set of attention vectors K , V .
3. These are used by each decoder to predict the output probabilities for each token in the sequence.
4. Repeat until a special symbol is reached.



<https://jalammar.github.io/illustrated-transformer/>

Decoder - Attention

- The self attention layer creates its Queries matrix from the layer below it.
- It then takes the K, V matrices from the output of the encoder stack.
- The self-attention layer is only allowed to attend to earlier positions in the output sequence.
- By **masking** future positions (-inf) before the softmax step in the attention calculation. The softmax will turn it to 0.

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

N

q1•k1	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞
q4•k1	q4•k2	q4•k3	q4•k4

N

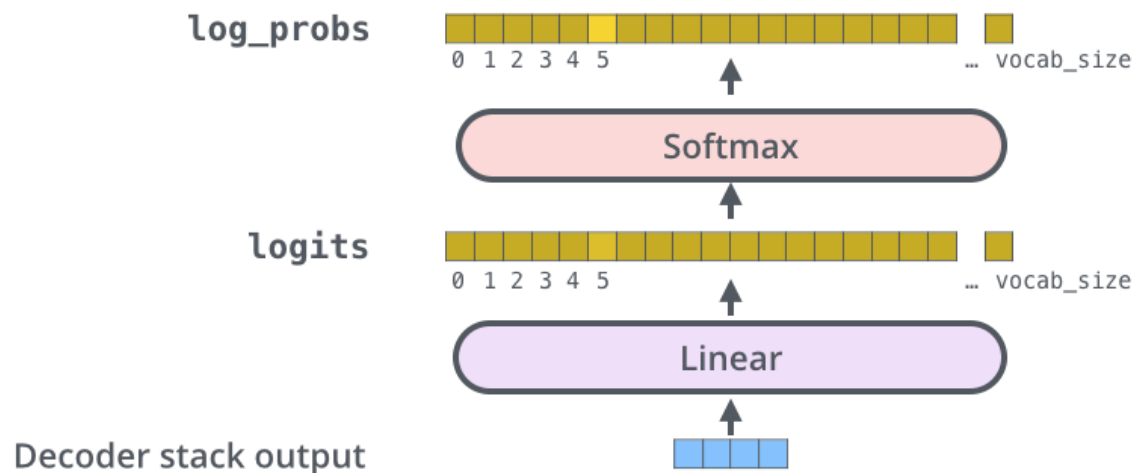
Decoder – Final linear and softmax layer

- Decoder returns a vector of float.
- The linear layer returns a logits vector (one value for each token in the model's vocabulary).
 - Similar to classification
- Softmax layer converts the logits into probabilities (all positive, add up to 1.).
- The cell with the highest probability is chosen, and the word associated with it is produced as the output for this timestep.

Which word in our vocabulary is associated with this index?

am

Get the index of the cell with the highest value (argmax)



<https://jalammar.github.io/illustrated-transformer/>

Results

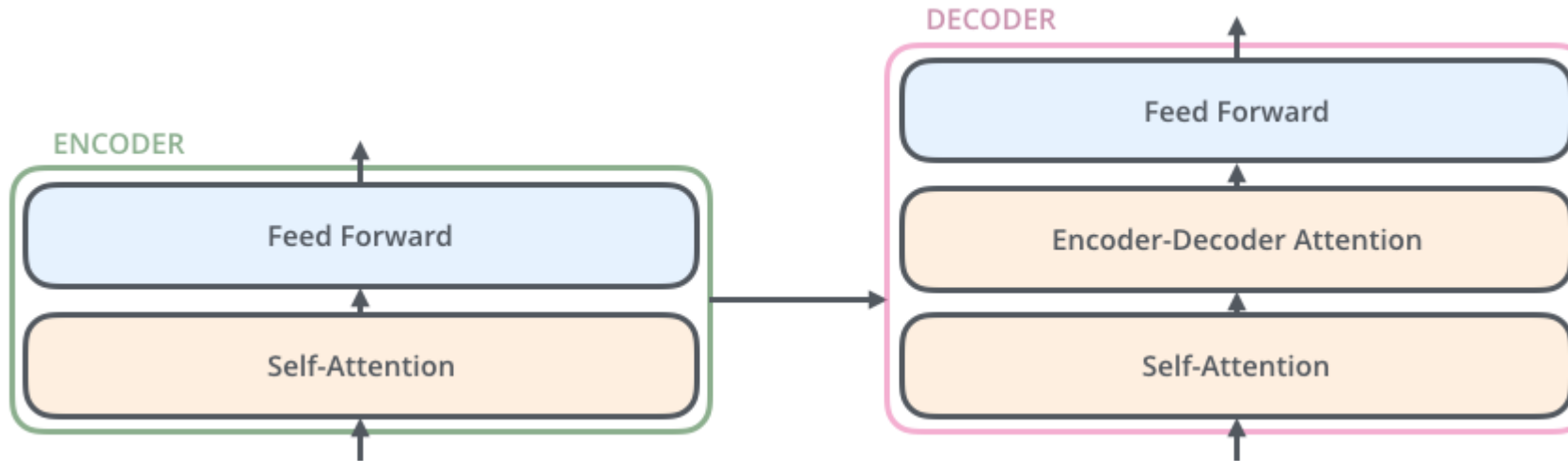
- At the time of release, single transformer models outperformed/matched the performance of ensemble of CNNs at an order of magnitude lower training cost.

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Architectural variants

► Standard transformer:



<https://jalammar.github.io/illustrated-transformer/>

Architectural variants – Vision Transformers

1. Input image (e.g., 224x224x3)
2. Break into patches (16x16x3)
3. Flatten and apply linear transform (768 $\rightarrow$ d)
4. D-dim vector per patch are the input vectors to the Transformer

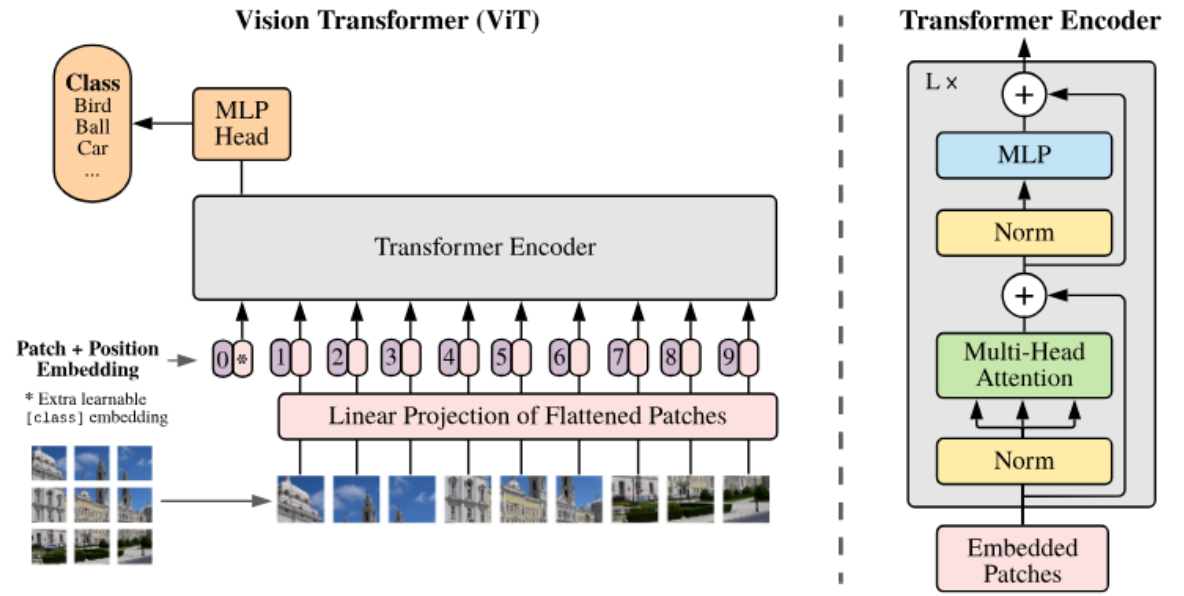
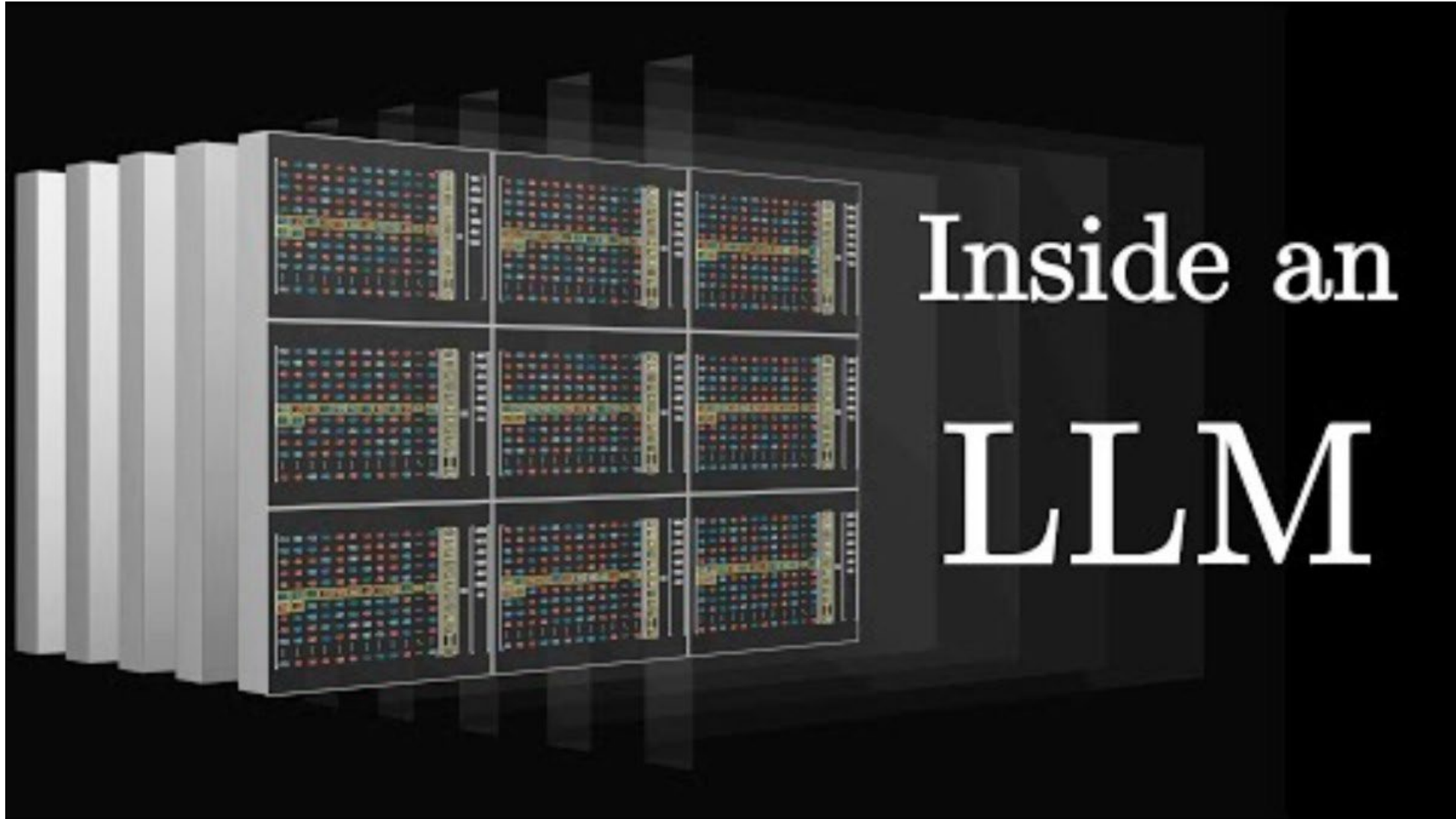


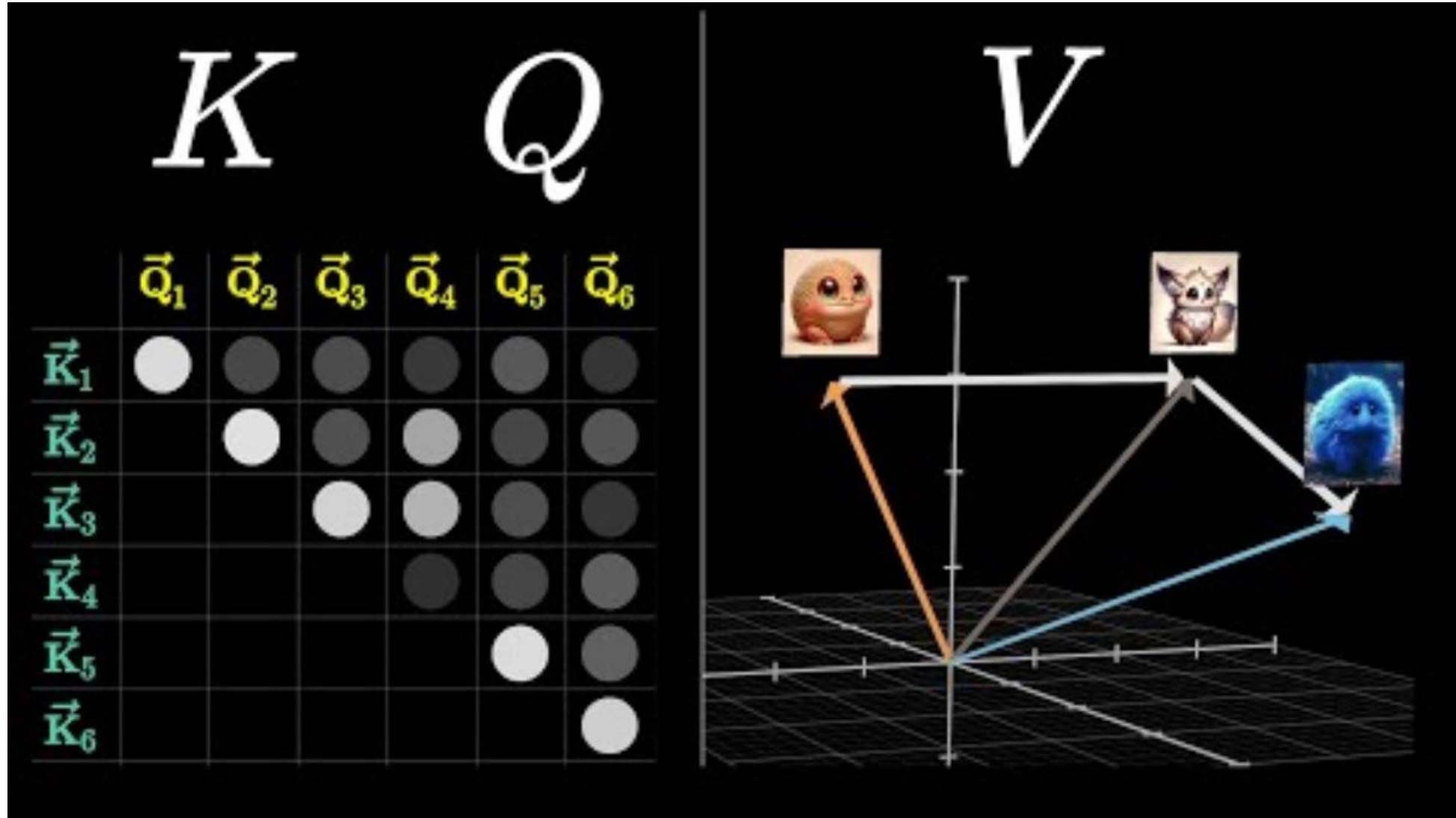
Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.

Some references

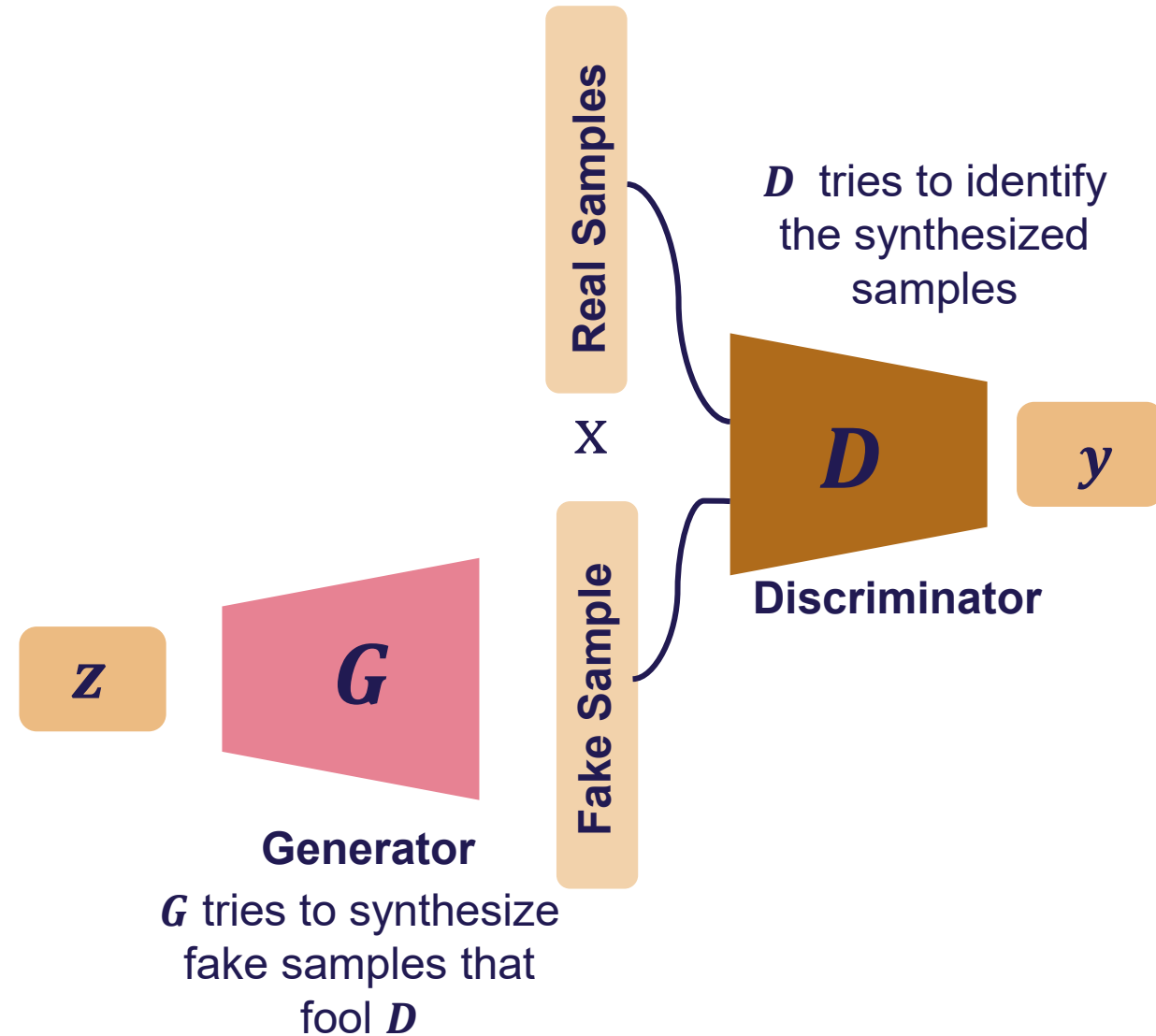


Some references



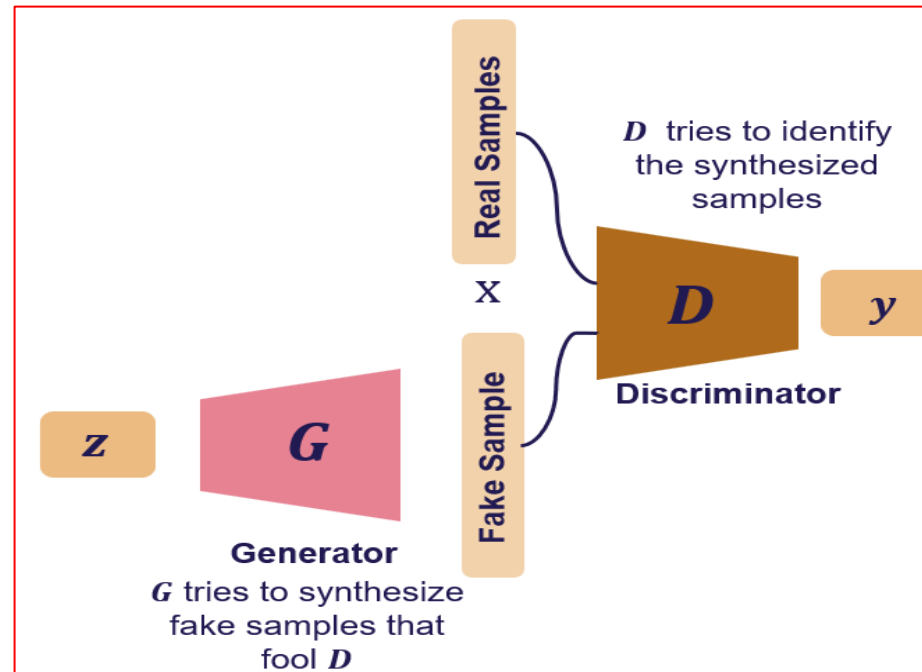
Generative Adversarial Networks (GANs)

- **Idea:** We don't explicitly model density, and instead just sample to generate new instances
- **Problem:** We want to sample from the complex distribution – but can't do this directly
- **Solution:** sample from something simple (e.g., noise), learn a transformation to the data distribution

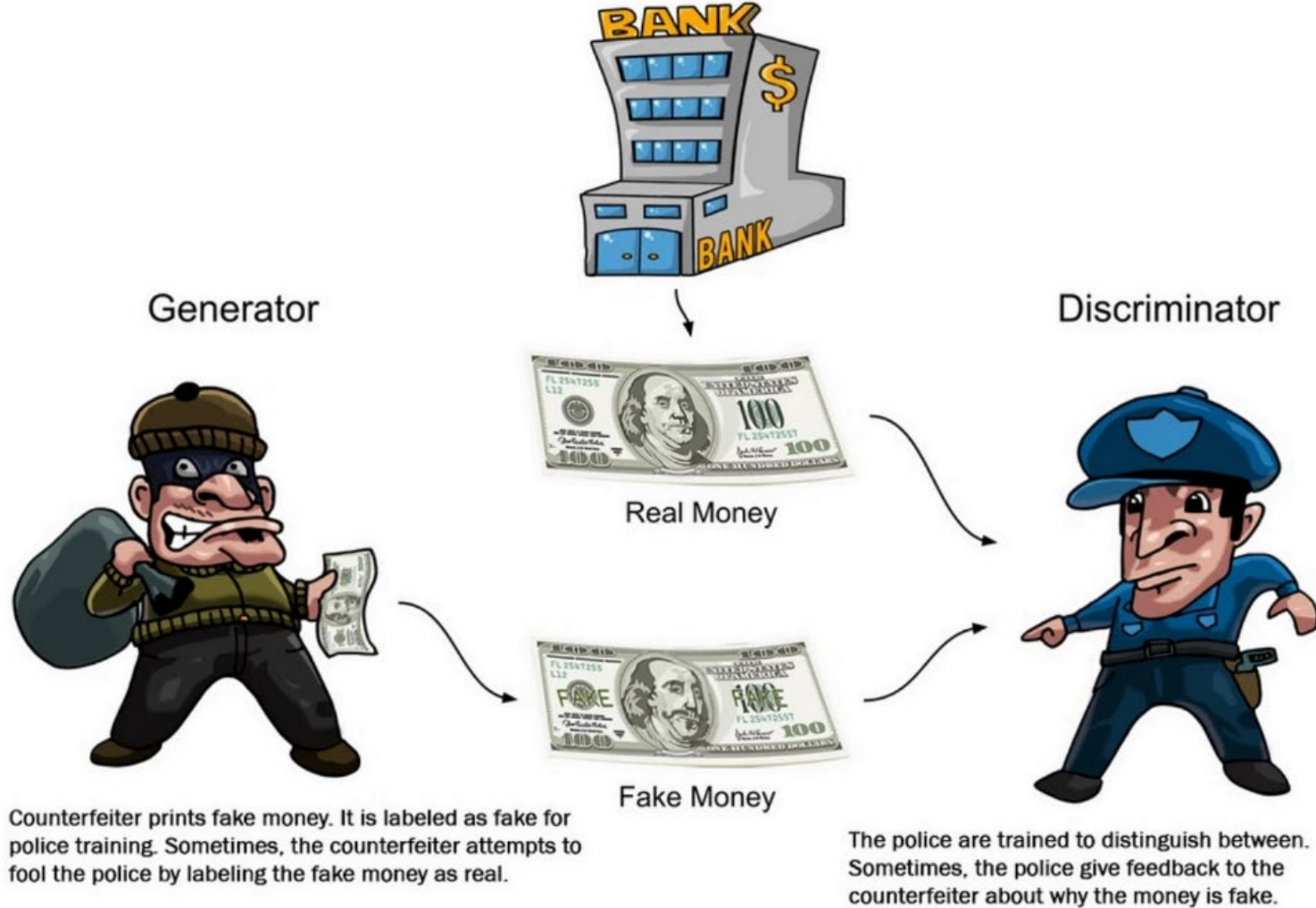


Generative Adversarial Networks (GANs)

- GANs is introduced by Ian Goodfellow and other researchers from the University of Montreal in 2014
- It is a neural network that belongs to the set of **generative AI**
- They are capable of generating new content that has never been generated before
- GANs fall under the category of unsupervised learning
- It can be applied to various domains such as images, text, audio. But it is commonly used in **images**
- The architecture consists of two neural networks: the **Generator** and the **Discriminator**

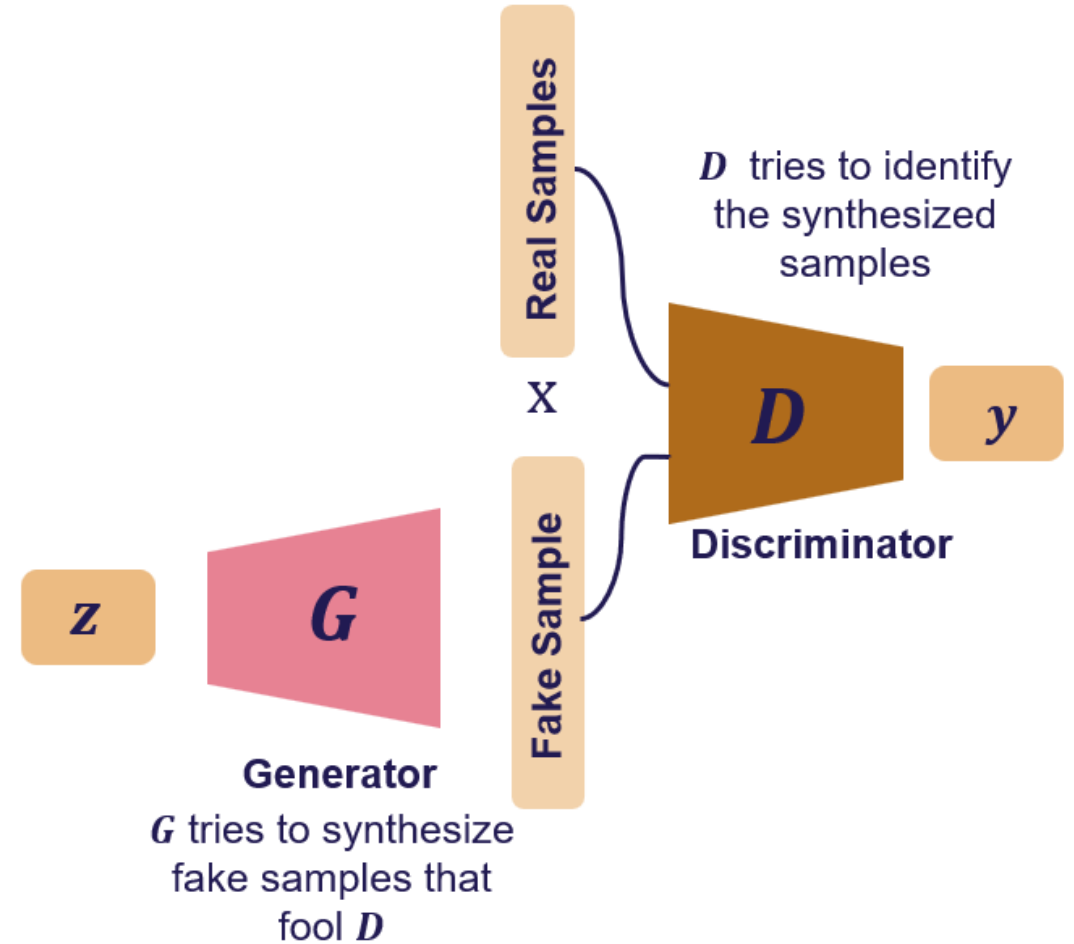


Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)

- The Generator considers random values to generate a fake sample – input (z)
- The generated sample is sent to the discriminator along with a stream of samples taken from the real data set
- The discriminator returns the probabilities of the generated image being real. The result is a number between 0 (fake) and 1 (real)

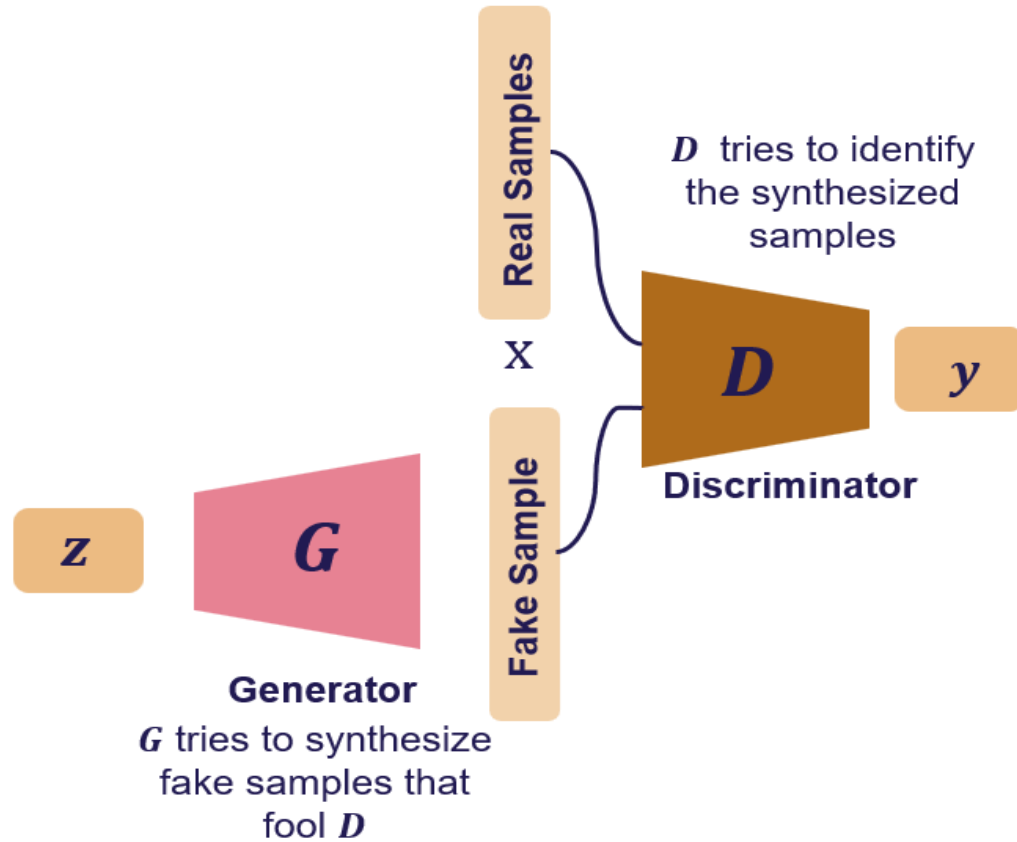


Generative Adversarial Networks (GANs)

- The Discriminator network is trained to distinguish between real and generated samples, playing an adversarial role with the generator
- It uses a loss function to measure the difference between real and fake data, guiding the training of the entire GAN system



Generative Adversarial Networks (GANs)



- The objective of the **Discriminator**:

$$\arg \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

- The objective of the **Generator**:

$$\arg \min_G \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

- The objective of the **GAN system**:

$$\arg \min_G \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

Major Types of GANs

- Vanilla GAN
- Deep Convolution GAN (**DCGAN**) (one of the most popular architectures)
- Conditional GAN (**cGAN**)
- Wasserstein GAN (**WGAN**)
- **StyleGAN**
- **CycleGAN**
- Super-Resolution GAN (**SRGAN**)
- **InfoGAN**
- **Pix2pix GAN**
- Progressive GAN (**ProGAN**)
- Self-Attention GAN (**SAGAN**)
- **BigGAN**
- **WaveGAN**

Applications

pix2pix



Super Resolution

upsampling a low-resolution image into a higher resolution



Fill missing parts of the image (inpainting)



Restoration of old images and noise removal



